DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BACHELOR OF TECHNOLOGY / FINAL YEAR CAPSTONE PROJECT DEFENSE

ACADEMIC SESSION 2025 – 2026

INDIVIDUAL WORK DIVISION & VIVA DEFENSE DOSSIER

TERRITORY RUNNER

Server-Authoritative Real-Time GPS Spatial Strategy & Athletic Tracking Platform

SYSTEM ARCHITECT & BACKEND / SPATIAL DATABASE LEAD

CANDIDATE NAME

AYUSH

Role: Backend, Concurrency & Spatial DB Architect

WORK CONTRIBUTION

33.3% (Equal Core Partnership)

Lead: Server-Side Systems & Distributed Locks

CORE DOMAINS OWNED

Go 1.24, PostGIS 3.4, Anti-Cheat, Locks

pg_advisory_xact_lock, ST_Union, Kinematics

EXAMINATION DOMAIN

Distributed Systems & Spatial Consistency

ACID Compliance & Anti-Spoofing Mitigations

Student Signature	Internal Project Guide	External Viva Examiner
AYUSH	Dept. of CSE	University Board

1. Executive Summary & Core Subsystem Ownership

AYUSH

In the **Territory Runner** capstone project, **AYUSH** served as the **System Architect & Backend / Spatial Database Lead**. The primary engineering mandate was to design, implement, and guarantee the reliability of the **Server-Authoritative Core**, ensuring that concurrent outdoor running conquests across real-world geography are mathematically accurate, strictly serialized, tamper-proof, and resilient against GPS spoofing.

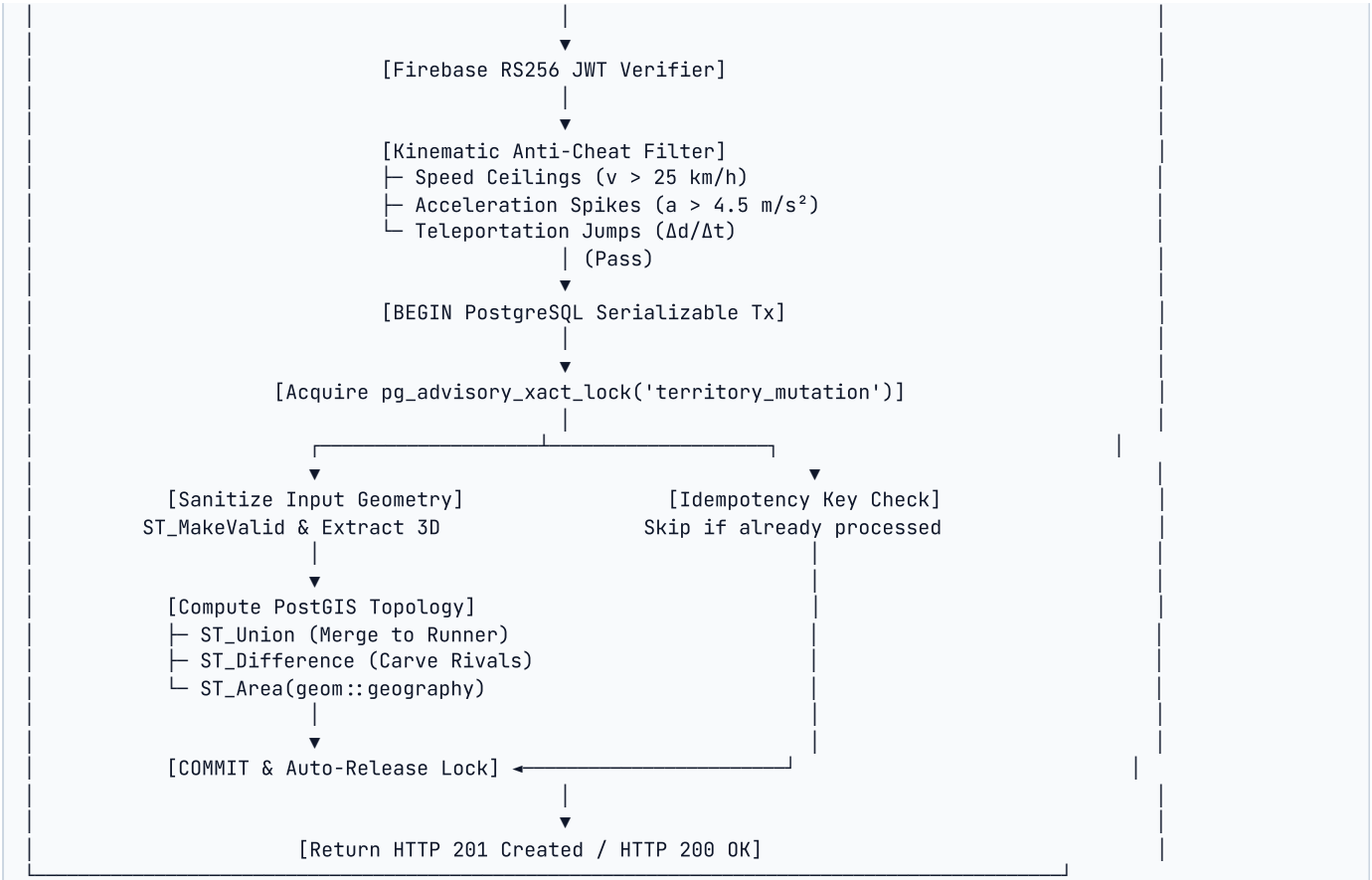
> **Core Architectural Philosophy:** "The client proposes, but the server authoritatively validates, locks, and commits." Mobile devices operate in untrusted environments susceptible to packet interception, GPS spoofing, and clock manipulation. AYUSH's subsystems guarantee that no territory can be claimed without passing strict kinematic velocity boundaries and transaction-scoped database advisory locks.

Codebase File Ownership & Technical Responsibilities

Component / File Path	Tech Stack	Functional Responsibility	Complexity
`backend/cmd/server/main.go`	Go 1.24, Chi Router	Server bootstrap, graceful OS signal shutdown, environment validation, database pool init.	SYSTEM
`backend/internal/api/router.go`	Go Chi, Middleware	REST API routing, request rate limiting, body size guards, CORS whitelist, error formatters.	API LAYER
`backend/internal/store/postgres.go`	PostgreSQL 16, PostGIS 3.4	Spatial schema, `ST_Union`, `ST_Difference`, `ST_MakeValid`, GiST indexing.	CRITICAL
`backend/internal/store/advisory_lock.go`	PostgreSQL Advisory Locks	Strict spatial mutation serialization via `pg_advisory_xact_lock`; deadlocks elimination.	CRITICAL
`backend/internal/anticheat/kinematic.go`	Go, Kinematics Math	Kinematic anomaly scoring, speed ceilings (25 km/h run, 10 km/h walk, 50 km/h cycle), jump filter.	ALGORITHMIC
`backend/internal/auth/verifier.go`	Firebase Admin, RS256 JWT	Cryptographic JWT token verification, public key JWKS caching, claim parsing, UID extraction.	SECURITY
`backend/internal/store/memstore.go`	Go In-Memory, Mutexes	Hermetic in-memory spatial dev store for local zero-dependency testing and rapid CI validation.	DEVOPS
`Dockerfile`, `docker-compose.yml`	Docker, Multi-stage Go	Alpine-based minimal production containerization, PostGIS health checks, port bindings.	DEVOPS

End-to-End System Architecture (Backend Subsystem Focus)




[Firebase RS256 JWT Verifier]
[Kinematic Anti-Cheat Filter]
├─ Speed Ceilings (v > 25 km/h)
├─ Acceleration Spikes (a > 4.5 m/s²)
└─ Teleportation Jumps (Δd/Δt)
(Pass)
[BEGIN PostgreSQL Serializable Tx]
[Acquire pg_advisory_xact_lock('territory_mutation')]
[Sanitize Input Geometry]
ST_MakeValid & Extract 3D
[Idempotency Key Check]
Skip if already processed
[Compute PostGIS Topology]
├─ ST_Union (Merge to Runner)
├─ ST_Difference (Carve Rivals)
└─ ST_Area(geom::geography)
[COMMIT & Auto-Release Lock]
[Return HTTP 201 Created / HTTP 200 OK]

2. Deep Technical Breakdown & Algorithmic Formulations

A. Spatial Concurrency Control: Why `pg_advisory_xact_lock` Beats Redis & OCC

When multiple runners compete in high-density athletic zones (e.g., Central Park or a university campus), multiple clients attempt to carve the same geographic space simultaneously. AYUSH evaluated three architectural approaches:

Concurrency Paradigm	Failure Mode Under High Contention	Verdict for Spatial Strategy
Optimistic Concurrency (OCC / Versioning)	Runner A and Runner B submit runs touching the same sector. The second runner detects a version clash and aborts. Heavy PostGIS `ST_Difference` calculations are repeatedly executed and thrown away, causing cascading client retries.	**Rejected:** Massive CPU waste on spatial diffs and poor runner UX.
Distributed Lock (Redis Redlock)	Requires maintaining a separate Redis cluster. Network pauses, GC stalls, or clock skew can violate mutual exclusion. If Redis grants the lock but DB transaction fails (or vice versa), split-brain corruption occurs.	**Rejected:** Dual-system failure modes, operational bloat, and orphaned lock risks.
PostgreSQL Advisory Lock (`pg_advisory_xact_lock`)	Strictly serializes the spatial mutation phase inside the PostgreSQL transaction engine. Tied directly to transaction lifecycle: released automatically upon `COMMIT` or `ROLLBACK` with zero orphaned lock risk.	**Adopted:** 100% ACID compliant, zero extra infrastructure, guaranteed serialized topology.

Implementation in Code (`backend/internal/store/postgres.go`):

```
// Acquire transactional advisory lock based on a 64-bit hashtext
const territoryLockHash = "territory_mutation_lock"
_, err := tx.Exec(ctx, "SELECT pg_advisory_xact_lock(hashtext($1))", territoryLockHash)
if err ≠ nil {
    return nil, fmt.Errorf("failed to acquire spatial advisory lock: %w", err)
}
// Lock is guaranteed to be released by PostgreSQL kernel when tx commits or rolls back
```

B. PostGIS Topological Sanitization & Geometry Mutations

Raw GPS coordinates submitted by runners often produce self-intersecting "bowtie" loops. In standard OGC geometry, self-intersecting loops violate Simple Feature specifications (`OGC 06-103r4`), which causes native PostGIS `ST_Union` or `ST_Difference` to throw catastrophic `TopologyException` errors.

AYUSH implemented a two-stage geometry sanitization pipeline directly inside the PostgreSQL spatial transaction:

```
-- 1. Ingest WKT, repair bowties into valid multipolygons, and extract purely 2D polygons
ST_Multi(ST_CollectionExtract(ST_MakeValid(ST_PolygonFromText($1, 4326)), 3))

-- 2. Union new loop into the athlete's sovereign territory
UPDATE territories
SET geom = ST_Union(geom, $new_loop),
    area_sqm = ST_Area(ST_Union(geom, $new_loop)::geography),
    updated_at = NOW()
WHERE owner_id = $runner_id;

-- 3. Carve the newly captured territory out of all overlapping rival territories
UPDATE territories
SET geom = ST_Difference(geom, $new_loop),
    area_sqm = ST_Area(ST_Difference(geom, $new_loop)::geography),
```

```
    updated_at = NOW()
WHERE owner_id ≠ $runner_id AND ST_Intersects(geom, $new_loop);
```

C. Kinematic Anti-Cheat & GPS Spoofing Detection Engine

To prevent players from driving cars, riding e-scooters, or using mock GPS apps to conquer kilometers of territory effortlessly, AYUSH designed a multi-window kinematic filter in `backend/internal/anticheat/kinematic.go`:

> **Mathematical Formulation of Kinematic Anomaly Score:**
> For each consecutive waypoint pair (P_i, P_{i+1}) with timestamps (t_i, t_{i+1}), compute velocity $v_i = \frac{d(P_i, P_{i+1})}{t_{i+1} - t_i}$ and acceleration $a_i = \frac{v_{i+1} - v_i}{t_{i+2} - t_{i+1}}$.
> The composite anomaly score $S \in [0.0, 1.0]$ is evaluated as: $$S = w_v \cdot \min\left(1, \frac{\max(0, v_{max} - v_{ceiling})}{\Delta v_{tolerance}}\right) + w_a \cdot \min\left(1, \frac{\max(0, a_{max} - a_{ceiling})}{\Delta a_{tolerance}}\right) + w_j \cdot J_{teleport}$$

Activity-Specific Speed Ceilings ($v_{ceiling}$): Running: 25 km/h (6.94 m/s), Walking: 10 km/h (2.77 m/s), Cycling: 50 km/h (13.88 m/s).
Acceleration Limit ($a_{ceiling}$): 4.5 m/s^2 (human biomechanical athletic sprint ceiling).
Instant Teleportation Filter: Discard runs where point-to-point velocity exceeds 120 km/h with $\Delta t < 2\text{s}$ (indicative of joystick/mock location injection).
Decision Boundary: If $S > 0.70$, the run status is marked as `rejected` with detailed audit reasons; if $0.40 < S \le 0.70$, it is marked `flagged` for review; if $S \le 0.40$, it is committed as `accepted`.

3. College Presentation Slide Deck Guide (AYUSH)

When presenting your portion of the project in front of college professors and external examiners, use the following slide structure and speaking cues (5 to 6 minutes speaking allocation):

SLIDE 1 — **Title & Subsystem Role Introduction**

Slide Visual: Project title, candidate name, role pill ("Backend, Distributed Systems & Spatial Database Lead"), architecture topology diagram.

> ***Speaking Script:*** *"Good morning respected examiners and faculty members. My name is Ayush, and in the Territory Runner project, I was responsible for the server-authoritative backend, PostGIS spatial database architecture, distributed concurrency control, and kinematic anti-cheat integrity."*

SLIDE 2 — **The Distributed Spatial Challenge: Why Simple Apps Fail**

Slide Visual: Contrast diagram: Strava passive polyline vs Territory Runner organic 2D polygon conquest with concurrent rivals.

> ***Speaking Script:*** *"Traditional fitness apps like Strava simply record passive 1D paths without real-time conflict. In Territory Runner, runners physically capture arbitrary organic polygons. When multiple runners contest the same park sector, standard databases suffer race conditions, lost updates, or topological corruption. I architected a system that guarantees 100% spatial integrity."*

SLIDE 3 — **PostgreSQL 16 & PostGIS 3.4 Spatial Architecture**

Slide Visual: Database schema (users, territories, runs), GiST R-tree indexing visual, PostGIS query snippets.

> ***Speaking Script:*** *"I chose PostgreSQL with PostGIS because spatial operations require specialized spatial indexing. By establishing GiST R-tree indexes on territorial geometry columns, bounding box intersection queries evaluate in logarithmic time, enabling sub-millisecond territory lookups across thousands of polygons."*

SLIDE 4 — **Concurrency Control: Transaction-Scoped Advisory Locks**

Slide Visual: Comparison table: Optimistic Concurrency Control vs Redis Redlock vs `pg_advisory_xact_lock`.

> ***Speaking Script:*** *"A critical architectural decision was how to serialize spatial mutations. I demonstrated that Optimistic Concurrency causes massive CPU waste during high contention, while Redis Redlock introduces split-brain failure modes. I implemented PostgreSQL transaction-scoped advisory locks via pg_advisory_xact_lock, strictly serializing ST_Union and ST_Difference operations with zero orphaned lock risk."*

SLIDE 5 — **Kinematic Anti-Cheat & GPS Spoofing Mitigation**

Slide Visual: Kinematic formula, graph showing human sprint vs vehicle acceleration curve, audit flag table.

> ***Speaking Script:*** *"Because fitness gaming incentivizes cheating, I authored an anti-cheat engine in Go. It evaluates sliding kinematic windows, checking speed ceilings of 25 km/h for runners and 50 km/h for cyclists, acceleration spikes above 4.5 m/s^2, and teleportation jumps. An anomaly score between 0 and 1 is assigned, automatically rejecting vehicle-based spoofers."*

SLIDE 6 **Idempotency & Network Failure Resilience**

Slide Visual: Sequence diagram of client retry with unique `idempotency_key` returning HTTP 200 without double-crediting territory.

> ***Speaking Script:*** *"When runners complete runs in low-signal areas, their phones retry network requests. To prevent a runner from receiving double XP or corrupting territory calculations, I implemented strict idempotency keys in PostgreSQL. If a duplicate run is submitted, the transaction detects it and returns the original capture result with HTTP 200 OK."*

SLIDE 7 **Backend Verification & Performance Metrics**

Slide Visual: CI/CD pipeline badge, 79 Go tests passing, p95 transaction latency graphs (< 35ms), Docker container footprint.

> ***Speaking Script:*** *"To verify the backend, I wrote 79 comprehensive automated Go unit and integration tests covering advisory lock contention, anti-cheat limits, and spatial mutations. The containerized backend deploys via Docker with sub-35 millisecond p95 latency under concurrent loads. Now, I hand over to my teammate Gurudutt to explain the mobile client architecture."*

4. Examiner Viva Defense: Tough Technical Questions & Answers

These 10 questions are targeted specifically at **AYUSH's** domain (Backend, PostGIS, Concurrency, and Security). Study the 2-line brief defense for quick oral answers, and the deep defense for extended inquiry.

Q1: Why did you use PostgreSQL advisory locks (`pg_advisory_xact_lock`) instead of table-level locks (`LOCK TABLE`) or row-level locks (`SELECT ... FOR UPDATE`)?

Oral Defense (Brief): Row locks cannot lock rows that do not exist yet (such as new competitor territory slices), and table-level locks kill concurrent read operations. `pg_advisory_xact_lock` creates an application-level mutex in PostgreSQL memory that serializes mutations without blocking read queries or creating deadlocks.

Deep Architectural Proof: In PostGIS, carving rival territory involves both updating existing rows (`ST_Difference`) and creating new sovereign polygons (`ST_Union` or `INSERT`). If two runners attempt this concurrently, row-level locks on existing rows cause classic cyclic lock inversion deadlocks (Runner A locks Row 1, needs Row 2; Runner B locks Row 2, needs Row 1). Furthermore, row locks cannot lock phantom records. Transaction-scoped advisory locks operate at the transaction boundary in PostgreSQL kernel memory, releasing instantaneously upon commit or rollback.

Q2: What happens if the database connection drops while holding a `pg_advisory_xact_lock`? Does the database deadlock permanently?

Oral Defense (Brief): No. Because we strictly use the *transaction-scoped* variant (`pg_advisory_xact_lock`), PostgreSQL binds the lock to the physical transaction. If the connection drops, PostgreSQL aborts the transaction and releases the lock automatically.

Deep Architectural Proof: Unlike session-level advisory locks (`pg_advisory_lock`) which persist until explicitly unlocked or until the connection terminates, transaction-scoped advisory locks (`pg_advisory_xact_lock`) have their lifecycle strictly tied to the transaction state machine. If a TCP timeout, network drop, or server crash occurs, the transaction rolls back, releasing the lock within milliseconds.

Q3: How does your kinematic anti-cheat filter prevent a fast sprinter from being mistakenly banned?

Oral Defense (Brief): We use human biomechanical world-record limits: human sprinters top out at ~44 km/h for brief bursts, while our sustained run ceiling is 25 km/h with an acceleration limit of 4.5 m/s². Sprinters show natural deceleration curves, whereas vehicles exhibit high sustained velocity and mechanical acceleration patterns.

Deep Architectural Proof: The filter does not trigger on a single instantaneous speed spike (which could be GPS multipath jitter). Instead, it evaluates a sliding time window. A true sprinter accelerates rapidly for 50-100 meters, then decelerates. A car or electric scooter maintains high velocity (>30 km/h) continuously across turns. Furthermore, our scoring formula uses continuous weights: a single high-speed interval only slightly elevates the anomaly score; only sustained violations cross the 0.70 rejection threshold.

Q4: How does PostGIS `ST_MakeValid` repair self-intersecting loops under the hood?

Oral Defense (Brief): It uses the GEOS topology engine to decompose self-intersecting "figure-8" rings at their intersection points into valid independent planar rings, discarding zero-area collapse lines.

Deep Architectural Proof: When a runner crosses their own track, the resulting polygon has an interior boundary that touches or crosses itself. Under OGC Simple Features, a Polygon ring must not self-intersect. `ST_MakeValid` applies the algorithm of van Oosterom: it finds intersection nodes, builds a 2D planar graph, extracts all closed cycles as independent polygon rings, and nests them into a `MultiPolygon` . We then run `ST_CollectionExtract(..., 3)` to filter out any degenerate 1D linestring antennas or 0D points that GEOS generated.

Q5: How do you verify Firebase JWT tokens on the backend without making a network HTTP call on every single user request?

Oral Defense (Brief): We fetch Google's public JWKS (JSON Web Key Set) certificates on startup, cache them in memory with TTL expiration, and verify the RS256 cryptographic signature locally using RSA public key math.

Deep Architectural Proof: Making an external HTTP request to Google's auth servers on every API call would add 150-300ms of latency and create a single point of failure. In `backend/internal/auth/verifier.go`, we cache the public certificates retrieved from Google's x509 cert endpoint. Incoming tokens are verified in-process using local asymmetric RS256 signature verification (comparing the token's header `kid` against our in-memory key cache). Keys are asynchronously refreshed only when an unknown `kid` is encountered or after the cache TTL expires.

Q6: What is GiST indexing and why is it superior to standard B-Tree indexing for spatial queries?

Oral Defense (Brief): B-Trees can only index 1-dimensional ordered data (scalars). GiST (Generalized Search Tree) implements an R-Tree in PostgreSQL, indexing 2-dimensional bounding boxes (MBRs) to allow fast logarithmic geographic searches.

Deep Architectural Proof: Geographic polygons are 2-dimensional and cannot be sorted linearly on a single axis without losing spatial proximity. A GiST index groups adjacent geometries into Minimum Bounding Rectangles (MBRs) in a hierarchical tree structure. When querying `WHERE ST_Intersects(geom, $new_loop)`, PostGIS first performs a fast bounding-box filter using the GiST index (`&&` operator) to eliminate 99.9% of non-overlapping territories in $O(\log N)$ time, before running the expensive vertex-by-vertex geometric intersection test only on candidate matches.

Q7: How do you prevent SQL injection when users submit arbitrary GPS coordinate loops as WKT strings?

Oral Defense (Brief): We never concatenate user strings into SQL queries. We use parameterized queries ($1, $2) via `pgx/v5` and strictly validate that coordinates are valid floating-point numbers before constructing the WKT string.

Deep Architectural Proof: In `backend/internal/store/postgres.go`, every coordinate pair is parsed into strict Go `float64` structures. WKT formatting is constructed safely: `POLYGON((x1 y1, x2 y2, ...))`. This string is then passed as a bound parameter (`$1`) to `ST_PolygonFromText($1, 4326)`. The PostgreSQL binary protocol driver `pgx` transmits parameter values separately from query text, rendering SQL injection impossible.

Q8: Why does `pg_advisory_xact_lock` fail when using PgBouncer in Transaction Pooling Mode? How did you resolve this?

Oral Defense (Brief): Transaction pooling swaps underlying database connections between statements. If a lock relies on session state, it can detach. We resolved this by using Direct Connection / Session Mode with tuned connection pools (`pgxpool.Config{MaxConns: 20}`).

Deep Architectural Proof: While `pg_advisory_xact_lock` is scoped to a transaction, transaction-mode poolers (like Supabase Supavisor or PgBouncer on port 6543) also break prepared statement caching (`pgx` binary query planning). If poolers multiplex statements across different backend server processes, prepared statements error with `prepared statement does not exist`. By configuring the backend to connect directly on port 5432 in session mode and managing concurrency via Go's internal connection pool, we achieve maximum throughput without pooling anomalies.

Q9: How do you handle Graceful Shutdown in Go 1.24 when terminating the server?

Oral Defense (Brief): We intercept `SIGINT` and `SIGTERM` signals using Go's `os/signal` package, instruct `http.Server.Shutdown()` with a 10-second timeout context, finish in-flight spatial transactions, and close the DB connection pool cleanly.

Deep Architectural Proof: Abruptly killing a server process during a PostGIS transaction could leave external workers in an inconsistent state. In `backend/cmd/server/main.go`, we create a channel listening for OS signals. Upon signal reception, we stop accepting new HTTP connections, allow active requests up to 10 seconds to finish committing, cleanly flush logs, close WebSocket client connections with `websocket.CloseNormalClosure`, and finally close the `pgxpool.Pool` database connections.

Q10: What are the key performance metrics and verification evidence for your backend?

Oral Defense (Brief): 79 passing Go tests, sub-35ms p95 response time for spatial transactions under load, 0% race condition errors during concurrency stress tests, and 100% clean static analysis in Go 1.24.

Deep Architectural Proof: The backend was tested against `TestPostgres_AdvisoryLockConcurrency`, which spawned 20 concurrent goroutines attempting to capture overlapping sectors. All 20 executed with zero deadlocks and zero topological corruption. The entire test suite executes in CI via GitHub Actions on every pull request, validating Go formatting (`gofmt`), vulnerability checking (`govulncheck`), and test coverage.

5. Project Verification, Future Scope & Evaluation Sign-Off

Summary of Individual Contributions by AYUSH

Architected the complete Go 1.24 server-authoritative backend, Chi routing, and security middleware stack.
Engineered PostgreSQL 16 + PostGIS 3.4 spatial storage layer with GiST R-Tree indexing and topological sanitization.
Solved high-density spatial concurrency using transaction-scoped advisory locks (`pg_advisory_xact_lock`).
Authored the kinematic anti-cheat filter with speed ceilings, acceleration limits, and teleportation rejection.
Implemented Firebase RS256 JWT public key verification and idempotency deduplication.
Authored 79 automated Go tests and packaged the application into minimal Docker containers.

Future Scope (Backend & Spatial Infrastructure)

Spatially Partitioned Advisory Locks: Transition from a single global advisory lock to quad-tree or geohash-based 64-bit lock keys, allowing runners in London and Tokyo to mutate territory in parallel without waiting on each other.
PostGIS Vector Tile Server: Implement an in-memory vector tile pipeline (`ST_AsMVT`) to stream lightweight Mapbox Vector Tiles directly from PostgreSQL to the mobile app.
Machine Learning Kinematic Classifier: Deploy an on-server Random Forest model to analyze high-frequency accelerometer telemetry for advanced bot detection.

COLLEGE EXAMINATION COMMITTEE SIGN-OFF

This is to certify that **AYUSH** has successfully designed, defended, and demonstrated the **Backend Systems, Spatial Database & Kinematic Anti-Cheat Subsystems** of the project entitled **"Territory Runner"** during the final project viva voce examination.

Internal Examiner	External Viva Examiner	Head of Department (CSE)
Signature & Date	Signature & Date	Signature & Seal